Pocket Smart AI – Your Smart Budget & Recommendation Assistant

Project Documentation

An AI-powered personal finance companion that tracks spending, builds smart budgets, and delivers personalized money-saving recommendations.

Document Type	Project Documentation
Application Category	AI / FinTech – Personal Budgeting
Platform	Web Application (React.js Frontend)
Version	1.0

TEAM MEMBERS	EMAIL ID
RAAHUL V	raahul1906a@gmail.com
ABDULLAH A	abdullahit.scas@gmail.com
SARATHI D	sarathiit.scas@gmail.com
GOWTHAM R	gowthamit.scas@gmail.com
SHURUTHI SAGAR R	shuruthisagarramesh@gmail.com

PROJECT OVERVIEW

Pocket Smart AI is an intelligent web application designed to help users take control of their personal finances effortlessly. It combines automated expense tracking, AI-driven budget planning, and personalized recommendations to help users save more, spend wisely, and reach their financial goals. The platform is built for everyone — from students managing pocket money to professionals planning long-term savings.

Purpose

Pocket Smart AI aims to make personal finance management simple, insightful, and proactive rather than reactive. The main goals are:

- Smart Budgeting – AI-generated monthly budgets based on income, spending habits, and goals.
- Personalized Recommendations – Actionable suggestions to cut costs and optimize savings.
- Effortless Expense Tracking – Automatic categorization of transactions and spending patterns.
- Modern Tech Stack – Built using React.js, Node.js, and a machine-learning recommendation engine.

Features

- AI Budget Engine – Generates a personalized monthly budget from income and expense data.
- Smart Recommendations – Suggests where to cut spending and how to reach savings goals faster.
- Expense Tracker – Logs and categorizes transactions automatically (food, bills, travel, etc.).
- Goal Planner – Lets users set savings goals and tracks progress in real time.
- Interactive Dashboards – Visual charts summarizing spending trends and budget health.
- Alerts & Insights – Notifies users when nearing budget limits or of unusual spending.

ARCHITECTURE

Component Structure

The application is divided into three main sections:

- Pages – Full-page views (Dashboard, Budget Planner, Recommendations, Reports).
- Components – Reusable UI elements (Navbar, Expense Cards, Charts, Goal Tracker).
- Styles – CSS and styling files for consistent theming.

State Management

- Global State – Managed using React Context API for user profile, budget data, and preferences.
- Local State – Controlled via React's useState for component-level interactions.

Routing: Implemented using React Router to enable seamless navigation between the Dashboard, Budget Planner, Recommendations, and Reports pages.

SETUP INSTRUCTIONS

Prerequisites

- Node.js & npm – Install from the official Node.js website.
- React.js – Set up a new project using create-react-app.

Installation Steps

- Clone the repository
- Navigate into the project directory
- Install dependencies – run npm install
- Set up environment variables by creating a .env file with the required API keys (e.g., recommendation engine key, currency API key).
- Start the development server – run npm start

Example commands:

```
git clone https://github.com/your-org/pocket-smart-ai
cd pocket-smart-ai
npm install
npm start
```

Once started, open `http://localhost:3000` in your web browser to access the application.

FOLDER STRUCTURE

The project is structured into different directories for better organization and maintainability.

```
pocket-smart-ai/
src/
components/ # Navbar, Cards, Charts, Modals
pages/ # Dashboard, Budget, Recommendations, Reports
context/ # BudgetContext.js, UserContext.js
services/ # API calls to the recommendation engine
styles/ # CSS / SCSS files
assets/ # Icons and images
public/
package.json
```

Client

The client folder (inside src/) contains the core files of the frontend application. It consists of:

- Components – Houses reusable UI components such as budget cards, charts, and the navigation bar.

- Pages – Contains full-page components, including the dashboard, budget planner, and reports page.
- Styles – Includes all CSS/SCSS files used to style the application.
- Assets – Stores icons, illustrations, and other static files used throughout the app.

RUNNING THE APPLICATION

To start the frontend server, run:

```
npm start
```

Then open `http://localhost:3000` in your browser to access the dashboard.

COMPONENT DOCUMENTATION

Key Components

Below are the major components of the Pocket Smart AI application, along with their purpose and props:

Navbar Component (Navbar.js)

Purpose:

- Provides site-wide navigation.
- Contains links to Dashboard, Budget, Recommendations, and Reports.

Props:

- logo (string): Path to the logo image.
- menuItems (array): List of navigation menu items.

Dashboard Summary Component (DashboardSummary.js)

Purpose:

- Displays an overview of income, expenses, and remaining budget.
- Shows quick-glance charts for the current month.

Props:

- income (number): Total monthly income.
- expenses (number): Total monthly expenses.
- budget (object): Current budget allocation by category.

Expense Card Component (ExpenseCard.js)

Purpose:

- Displays a single transaction with category, amount, and date.

- Allows users to edit or delete an entry.

Props:

- expense (object): Contains id, title, amount, category, and date.

Recommendation Card Component (RecommendationCard.js)

Purpose:

- Displays an AI-generated saving tip or budget adjustment suggestion.
- Lets users accept or dismiss a recommendation.

Props:

- recommendation (object): Contains id, message, category, and potential savings.

Budget Planner Component (BudgetPlanner.js)

Purpose:

- Allows users to set or adjust category-wise budget limits.
- Visualizes allocation versus actual spending.

Props:

- categories (array): List of budget categories and limits.

Goal Tracker Component (GoalTracker.js)

Purpose:

- Displays progress toward a user-defined savings goal.

Props:

- goal (object): Contains name, targetAmount, and currentAmount.

Reusable Components

Search / Filter Bar Component (FilterBar.js)

Purpose:

- Allows users to filter transactions by category, date, or amount.

Props:

- onFilter (function): Callback function for handling filter changes.

Button Component (Button.js)

Purpose:

- A reusable button component with customizable styles.

Props:

- text (string): Button label.
- onClick (function): Click event handler.
- variant (string): Defines button style (e.g., primary, secondary).

Loading Spinner Component (Loading.js)

Purpose:

- Displays a loading animation while fetching data or generating recommendations.

Props:

- size (string): Size of the spinner (small, medium, large).

STATE MANAGEMENT

State management in the Pocket Smart AI application ensures efficient data handling across different components. The project incorporates both global state for shared data and local state for component-specific interactions.

Global State

The application uses the React Context API to manage global state, allowing components to access shared data without prop drilling.

- Budget Data Storage – Stores income, expenses, and category budgets.
- Recommendation State – Manages AI-generated suggestions and their status (accepted/dismissed).
- User Preferences – Keeps track of savings goals and notification settings.

How global state flows across the application:

- The BudgetContext.js file initializes and manages the global state.
- The BudgetProvider component wraps the application to provide access to the global state.
- Components like Dashboard.js and Recommendations.js consume global state using useContext.
- Any update to the global state (e.g., a new transaction) automatically reflects across all dependent components.

Local State

Local state is managed using the useState hook within individual components for temporary UI interactions that don't need to persist across the application.

- Filter Bar – Stores and updates the user's active filter selection.
- Budget Planner – Keeps track of unsaved changes to category limits.

- UI Components – Controls modals, dropdown menus, and loading states.

USER INTERFACE

The Pocket Smart AI application features a clean, data-forward interface designed to make financial insights easy to understand at a glance.

Dashboard (Overview & Quick Insights)

Features:

- A summary banner showing income, expenses, and remaining budget.
- Visual charts of spending by category.
- A quick-access panel for the latest AI recommendations.

Budget Planner Page

Features:

- Category-wise budget allocation with sliders and input fields.
- Progress bars comparing planned versus actual spending.
- One-click option to apply AI-suggested budget adjustments.

Recommendations Page

Features:

- Lists personalized saving tips generated by the AI engine.
- Each card shows potential monthly savings and a short explanation.
- Users can accept, dismiss, or save a recommendation for later.

Reports & Analytics Page

Features:

- Monthly and yearly spending trends in chart form.
- Breakdown of expenses by category and merchant.
- Exportable summaries for personal record-keeping.

Responsive UI & Mobile View

Features:

- The application is fully responsive across different screen sizes.
- Mobile-friendly navigation menu and swipeable transaction cards.

STYLING

Pocket Smart AI is designed with a modern, calm, and trustworthy aesthetic appropriate for a finance application, using industry-standard styling techniques.

CSS Frameworks & Libraries Used

- Tailwind CSS – A utility-first framework used for fast, consistent, and responsive styling.
- Recharts / Chart.js – Used to render budget and spending visualizations.
- React Icons – Used for navigation, category, and status icons throughout the UI.

Theming & Custom Design

- Consistent Color Scheme – A calm green-and-neutral palette conveys trust and financial clarity.
- Primary Color – Used for buttons, highlights, and key call-to-action elements.
- Secondary Color – Used for backgrounds and supporting UI elements.
- Dark Mode & Light Mode – Users can toggle between modes to reduce eye strain.

Animations & Transitions

- Smooth chart animations when budget data updates.
- Fade-in transitions for newly generated AI recommendations.
- Lazy loading for report data to keep the app lightweight and fast.

TESTING

Ensuring the reliability and accuracy of financial calculations and recommendations is critical. The testing approach includes multiple strategies.

Testing Strategy

Unit Testing:

- Used Jest and React Testing Library to test individual components.
- Verified budget calculations and category totals render correctly.

Integration Testing:

- Verified that the recommendation engine returns and displays suggestions correctly.
- Ensured that updating a transaction refreshes the dashboard and reports in real time.

End-to-End Testing:

- Used Cypress to simulate real user workflows, including adding an expense, setting a budget, and reviewing a recommendation.

Code Coverage & Goals

- Achieve 80%+ test coverage, focusing on budget calculations, recommendation logic, and UI rendering.
- Implement CI/CD pipelines to run tests automatically before deployment.

KNOWN ISSUES

Despite extensive testing, the Pocket Smart AI application may have some known issues that could affect user experience.

Slow Recommendation Generation

Issue:

- Recommendation data occasionally takes longer to load when the AI engine is processing a large transaction history.

Possible Cause:

- High volume of transactions or network latency.

Potential Solution:

- Implement caching for recent recommendations and show a loading indicator while processing.

Category Misclassification

Issue:

- Some transactions are placed in the wrong spending category.

Possible Cause:

- The auto-categorization logic does not always match uncommon merchant names.

Potential Solution:

- Allow users to manually recategorize transactions, feeding corrections back into the model.

UI Overlapping on Mobile Screens

Issue:

- Some UI components (e.g., charts, navbar) overlap on smaller screens.

Possible Cause:

- Missing proper media queries for responsive design.

Potential Solution:

- Adjust CSS breakpoints and use flexbox/grid layouts for dynamic positioning.

No Feedback on Failed API Calls

Issue:

- When the recommendation API fails, users don't receive an error message.

Possible Cause:

- No error handling implemented in API calls.

Potential Solution:

- Implement try-catch blocks and display fallback UI with a retry option.

These issues will be addressed in future updates to enhance the performance, usability, and reliability of the Pocket Smart AI application.

FUTURE ENHANCEMENTS

Pocket Smart AI is designed to provide a seamless experience for budgeting and financial planning. Below are potential future enhancements.

User Authentication & Multi-Device Sync

Enhancement:

- Implement secure sign-up/login and sync budget data across devices.

Benefit:

- Enables users to access their financial data anywhere, securely.

Bank Account Integration

Enhancement:

- Connect directly to bank accounts for automatic transaction import.

Benefit:

- Removes the need for manual entry and improves data accuracy.

Predictive Spending Alerts

Enhancement:

- Use AI to forecast upcoming expenses and warn users before overspending.

Benefit:

- Helps users stay ahead of their budget instead of reacting after the fact.

Bill Reminder & Auto-Categorization

Enhancement:

- Automatically detect recurring bills and remind users before due dates.

Benefit:

- Reduces missed payments and late fees.

Investment & Savings Suggestions

Enhancement:

- Recommend savings or investment options based on surplus income.

Benefit:

- Helps users grow wealth beyond basic budgeting.

Voice-Activated Finance Assistant

Enhancement:

- Integrate voice commands for quick expense logging and balance checks.

Benefit:

- Makes daily tracking faster and more convenient.